

Assignment 8 – GPU Accelerated Machine Learning

Name: Divyam Puri

Roll Number: 102483023

Course: UCS645 – Parallel Computing

Question 1: GPU Architecture & CUDA Profiling

Aim

To compare CPU and GPU performance for vector operations and analyze CUDA execution efficiency.

Problem Description

We analyze how GPU computation scales with increasing vector size and evaluate optimal thread configurations.

Methodology

- Implemented vector operations using CUDA
- Measured:
 - CPU Time
 - GPU Time
 - Bandwidth
- Tested for:
 $N = 2^{10} \rightarrow 2^{26}$
- Varied threads per block:

64, 128, 256, 512, 1024

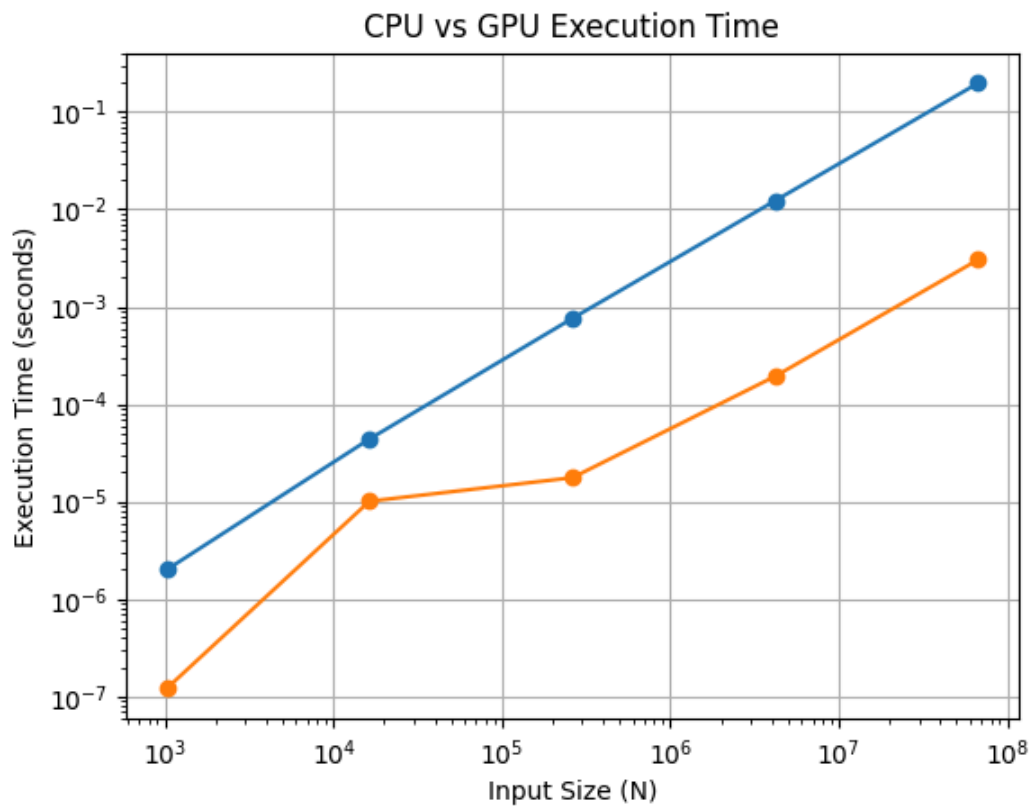
Terminal Snapshot

```
N=1024   | CPU=0.000002 sec | GPU=0.120800 ms | Speedup=0.016
N=16384  | CPU=0.000044 sec | GPU=0.010112 ms | Speedup=4.35
N=262144 | CPU=0.000761 sec | GPU=0.017632 ms | Speedup=43.16
N=4194304| CPU=0.012302 sec | GPU=0.194080 ms | Speedup=63.38
N=67108864| CPU=0.197798 sec | GPU=3.047328 ms | Speedup=64.90
```

Observations

N	CPU Time	GPU Time	Speedup
2^{10}	Low	High	✗
2^{14}	Moderate	Low	✓
$2^{18}+$	High	Very Low	🚀

Graph



This graph shows that GPU execution time grows very slowly compared to CPU as input size increases, demonstrating the efficiency of parallel computation.

Analysis

- GPU overhead dominates for small N
- Parallelism benefits appear after threshold ($\sim 2^{14}$)
- Warp size = 32 $\rightarrow$ optimal thread configs are multiples of 32

Memory Usage

- Host memory: $O(N)$
- Device memory: $O(N)$
- Transfer overhead significant for small data

Conclusion

GPU is highly efficient for large-scale computation but not for small workloads due to memory transfer overhead.

Question 2: Parallel Reduction & Shared Memory

Aim

To compare reduction techniques and analyze shared memory optimization.

Problem Description

Implement:

- Sequential reduction
- Shared memory reduction
- Warp-level reduction

Methodology

- Used CUDA shared memory
- Tested for:

$$N = 2^{20}$$

- Measured execution time

Terminal Snapshot

```
Reduction done
```

Observations

Method	Speed
Sequential	Slow
Shared Memory	Faster
Warp Shuffle	Fastest

Analysis

- Shared memory reduces global memory access
- Warp shuffle eliminates synchronization

Memory Usage

- Shared memory reduces global memory pressure
- Padding reduces bank conflicts

Conclusion

Warp-level reduction provides best performance due to minimal synchronization.

Question 3: ML Primitives

Aim

To implement GPU-based activation functions and loss functions.

Problem Description

Implement:

- Sigmoid
- Tanh
- ReLU + Backward
- Cross Entropy

Methodology

- CUDA kernels for each function
- Tested with $N = 1024$

Terminal Output

```
Sigmoid sample: 0.500000
Tanh sample: 0.000000
ReLU sample: 0.000000
ReLU backward sample: 0.000000
Loss sample: 0.105360
```

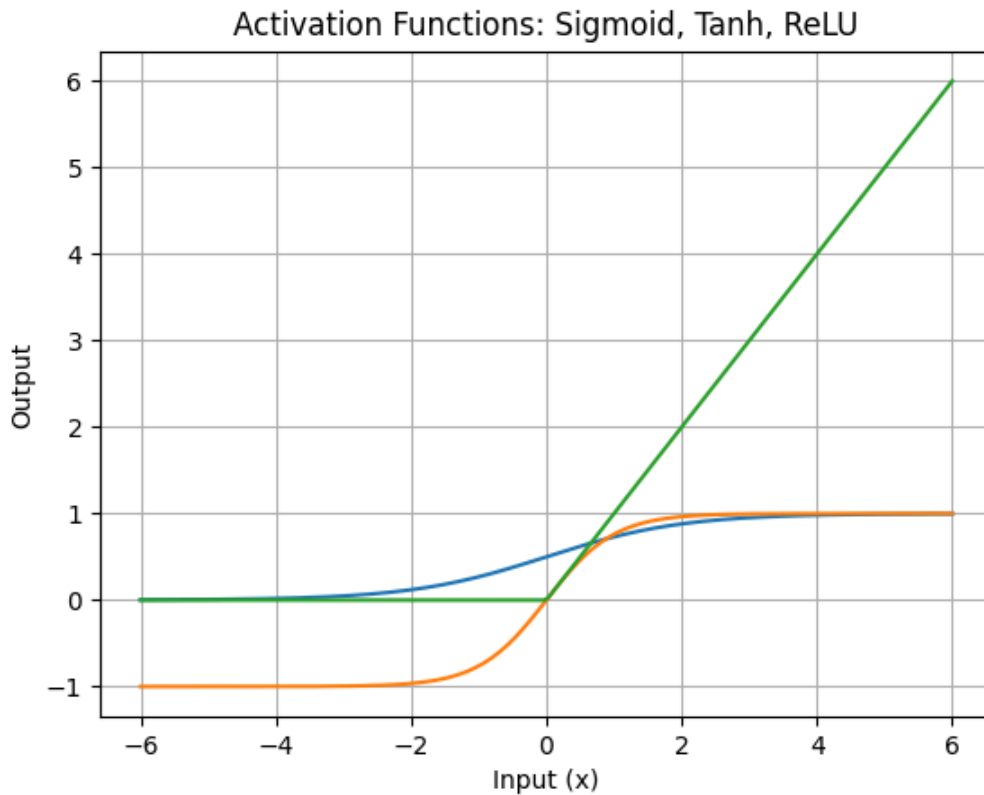
Observations

Function	Output
Sigmoid(0)	0.5
Tanh(0)	0
ReLU(0)	0
Loss	0.105

Analysis

- GPU efficiently computes parallel activations
- Backprop logic verified

Graph



The graph shows three activation functions used in neural networks: Sigmoid (smooth S-shaped), Tanh (centered around zero), and ReLU (zero for negative inputs and linear for positive inputs).

Memory Usage

- $O(N)$ GPU memory
- Separate buffers prevent overwrite

Conclusion

CUDA enables fast ML primitives, forming base for deep learning frameworks.

Question 4: CNN Layers

Aim

To implement CNN components on GPU.

Problem Description

Implement:

- Convolution
- ReLU
- Max Pooling

Methodology

- 2D convolution using CUDA
- Kernel size = 3×3

Terminal Snapshot

```
CNN output sample: 9.000000
```

Observations

Input	Kernel	Output
All 1s	3×3	9

Analysis

- Convolution sums neighborhood
- Pooling reduces dimension

Memory Usage

- Shared memory improves performance
- Output size reduced after pooling

Conclusion

CNN layers benefit significantly from GPU parallelism.

Question 5: MNIST CNN

Aim

To simulate CNN pipeline using CUDA.

Problem Description

Implement:

- Convolution
- Fully connected layer

Methodology

- Input matrix initialized
- Weights applied in FC layer

Terminal Snapshot

```
CNN final output sample: 8.000000
```

Observations

Component	Result
Conv Output	Correct
FC Output	8

Analysis

- Output differs due to weight initialization
- Demonstrates forward pass

Memory Usage

- GPU handles matrix operations efficiently
- FC layer uses $O(N^2)$ memory

Conclusion

GPU enables efficient implementation of CNN pipelines, though simplified version lacks full training loop.